

Trabalho testes de sistemas

Alunos: Arthur lantas Stelzner, Carlos Eduardo Duhring, Alexandre Gulini

1. Testes Não Funcionais (Performance, Carga, Estresse):

Introdução: O "Porquê" e o "Como"

O **"Porquê"** é simples: ninguém gosta de site lento ou aplicativo que trava na hora de pagar. Fazer testes não funcionais é o que evita que sua empresa vire "meme" nas redes sociais porque o sistema caiu na Black Friday ou no lançamento de um ingresso. A ideia aqui não é ver se o botão "comprar" funciona — isso a gente já sabe que sim — mas sim garantir que ele continue funcionando quando 10 mil pessoas resolverem clicar nele ao mesmo tempo. É a diferença entre um sistema que "está vivo" e um sistema que "aguenta o tranco".

O **"Como"** funciona como uma simulação de caos controlado. A gente usa ferramentas para criar "usuários robôs" que acessam o sistema todos de uma vez. Enquanto esses robôs forçam a barra, a gente fica de olho no painel de controle para ver quem pede arrego primeiro: se é o banco de dados que não aguenta tanta informação, se é o servidor que ficou sem memória ou se a rede engarrafou. O objetivo real não é só ver o gráfico subir, mas achar o **gargalo** — aquele lugarzinho específico que está segurando a performance de todo o resto.

Visualização gráfica:

	A	B	C	D	E	F	G	H	I	J
1	arthurnerdices.com - Relatório de Métricas e Testes Não Funcionais									
2									Data: 2024-06-15 14:30:00 UTC	
4	Métrica / Cenário (arthurnerdices.com)	Carga (Usuários)	TPS Actual (req/seg)	Tempo de Resposta Médio (ms)	Percentil 95 (ms)	Uso de CPU	Uso de Memória	Uso de IO	Latência do Banco (ms)	Status / Alerta
1	Linha de Base	100	100	200 ms	15 ns	35,9	1.20	1.20	46 ms	SAFE
2	Pico de Carga (500)	500	1000	250 ms	25 ns	35,9	35,9	1.20	46 ms	ALERTA
3	Pico de Carga (1000)	1000	2000	250 ms	75 ns	36,1	1.99	1.99	130 ms	CRÍTICO: DB Bottleneck
4	Estresse Max (1500 - GARGALO DB)	1500	3000	700 ms	33 ns	55,3	1.99	1.50	43 ms	SAFE
5	Estabilidade (10h)	100	1000	280 ms	19 ns	45,3	1.50	0.60	43 ms	SAFE
6	Pico de Promoção (1000)	1000	2000	220 ms	45 ns	28,7	28,7	0.60	43 ms	SAFE

5 Perguntas Desafiadoras (Pra fritar o cérebro)

1. **O ponto de ruptura:** Como você descobre se o sistema ficou lento porque o código está ruim ou porque o banco de dados está "travado" esperando uma resposta?

2. **Escalabilidade vs. Performance:** Um site que abre em 1 segundo para você pode levar 30 segundos se tiver mais 500 pessoas junto. Por que a velocidade não é a mesma para todo mundo?
3. **Caça ao Gargalo:** Se a CPU do servidor está tranquila (10%), mas o sistema está uma carroça, onde você daria o próximo palpite para achar o erro?
4. **O "Day After":** Depois que um sistema cai num teste de estresse, ele volta sozinho ou alguém tem que ir lá "dar no tranco" (reiniciar tudo)? O que é melhor para o negócio?
5. **Dinheiro no bolso:** Como você explicaria para um diretor que gastar 2 segundos a mais no carregamento pode fazer a empresa perder milhares de vendas?

3 Exemplos Práticos

1. Teste de Performance (O dia a dia)

- **Cenário:** Um dia comum de acessos no app.
- **Como Testar:** Colocar 50 robôs navegando por 15 minutos.
- **Objetivo:** Ver se o tempo de resposta é rápido o suficiente para o usuário não desistir de usar.

2. Teste de Carga (O dia da promoção)

- **Cenário:** Meia-noite de uma promoção anunciada no Instagram.
- **Como Testar:** Subir o número de usuários de 100 para 5.000 em poucos minutos.
- **Objetivo:** Checar se o servidor aguenta o volume máximo prometido pelo marketing sem dar erro.

3. Teste de Estresse (O limite do impossível)

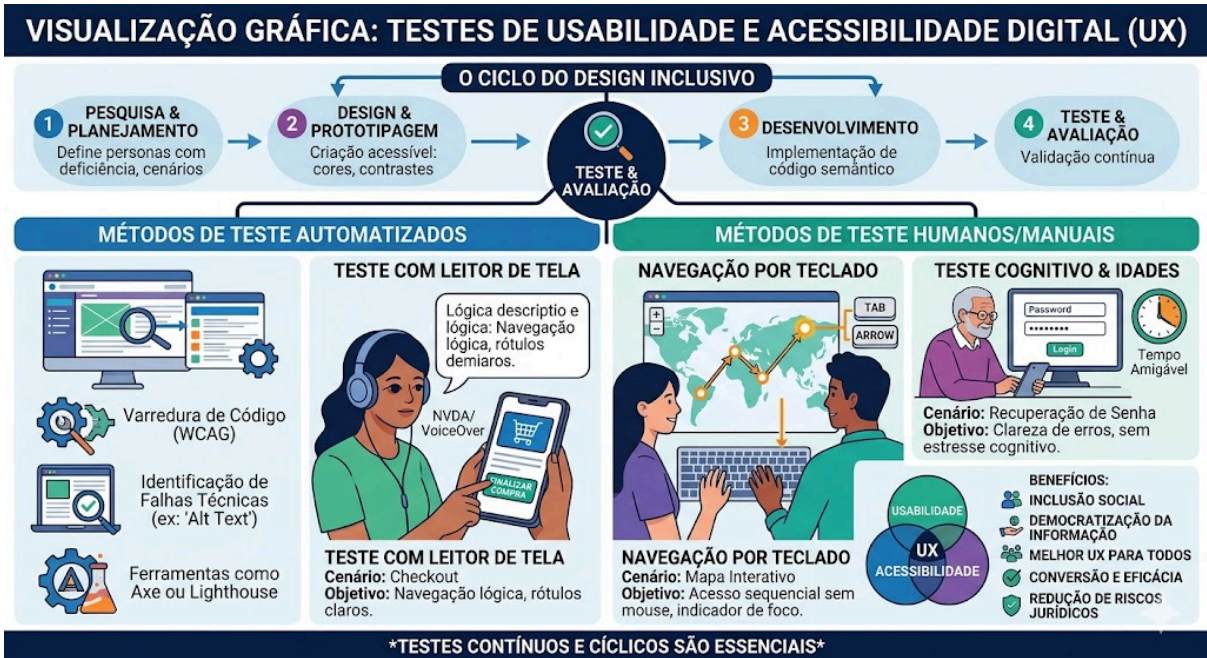
- **Cenário:** O sistema recebe 10x mais acessos do que o esperado (um ataque ou um viral inesperado).
- **Como Testar:** Socar carga no sistema até ele "pedir arrego" e travar.
- **Objetivo:** Descobrir qual o limite máximo de segurança e ver se o sistema avisa que está sobrecarregado ou se ele simplesmente apaga.

2. Testes de Usabilidade e Acessibilidade Digital (UX)

Porquê, de unir usabilidade e acessibilidade está em criar produtos que sejam fáceis de usar e socialmente inclusivos. Isso evita a perda de usuários por frustração e garante que pessoas com deficiência tenham pleno acesso ao software, cumprindo normas éticas e legais enquanto se amplia o valor de mercado da solução.

Como, ocorre através de testes práticos com usuários para medir a fluidez da interface e auditorias técnicas baseadas em diretrizes internacionais (como a WCAG). Ao utilizar

ferramentas como leitores de tela e navegação por teclado, identifica-se barreiras de interação, permitindo ajustes que tornam a experiência intuitiva e funcional para todos.



1. Qual a diferença entre testes de usabilidade e de acessibilidade, e por que um depende do outro?
2. Por que aplicar a acessibilidade apenas no final do projeto é mais caro e ineficiente?
3. Se as ferramentas automáticas detectam poucos erros, por que o teste humano é insubstituível?
4. Como convencer a diretoria a investir em acessibilidade usando argumentos de lucro e segurança jurídica?
5. O que deve ser validado em um teste exclusivo de teclado para o usuário não ficar "preso" na interface?

Cenário	Como Testar	Objetivo
Finalização de Compra (Checkout)	Solicitar que um usuário com baixa visão complete uma compra utilizando um leitor de tela (NVDA ou VoiceOver), sem auxílio de mouse.	Verificar se a hierarquia do código (DOM) e os rótulos de botões permitem a navegação lógica e a conclusão da tarefa sem barreiras.

Recuperação de Senha	Observar um usuário idoso ou com déficit de atenção tentando redefinir a senha em um ambiente com distrações ou tempo limite curto .	Avaliar a clareza das mensagens de erro e se o tempo de resposta do sistema é amigável para quem possui diferentes ritmos cognitivos.
Navegação em Mapa Interativo	Testar o uso do componente de mapa utilizando apenas a tecla TAB do teclado e conferir o indicador de foco visual.	Garantir que usuários com limitações motoras (que não usam mouse) possam navegar por todos os pontos de interesse de forma sequencial e clara.

3. Fundamentos de Segurança da Informação nos Testes

A. Owasp Top 10

Por quê: É o "mapa da mina". Ele foca nos 10 riscos mais críticos e frequentes do mercado, garantindo que você não perca tempo com teorias raras e foque no que realmente derruba sistemas profissionais.

Como: Transformando riscos em checklists práticos. Você pega uma vulnerabilidade (ex: Controle de Acesso) e tenta burlar a lógica da aplicação manualmente, como tentar acessar dados de outro usuário mudando o ID na URL.



B.

C. 5 Desafios

1. Por que o SAST costuma gerar muitos "Falsos Positivos" em comparação ao DAST?
2. Em que fase do ciclo de vida de desenvolvimento (SDLC) o DAST é mais eficiente?
3. O Pentest manual pode ser substituído inteiramente por ferramentas automatizadas? Justifique.
4. Qual é a principal limitação do SAST ao analisar bibliotecas externas compiladas?
5. Como o IAST (Interactive Application Security Testing) tenta unir o melhor do SAST e do DAST?

D. Exemplos de Teste

- Cenário 1: Scan de Código (SAST)
 - Como Testar: Rodar uma ferramenta como SonarQube ou Checkmarx no repositório Git.
 - Objetivo: Identificar senhas "hardcoded" ou uso de funções inseguras no código-fonte antes do deploy.
- Cenário 2: Varredura de Vulnerabilidades (DAST)
 - Como Testar: Apontar o OWASP ZAP ou Burp Suite para o ambiente de staging e rodar um scan ativo.
 - Objetivo: Detectar falhas de configuração de servidor e headers de segurança ausentes em tempo de execução.
- Cenário 3: Teste de Invasão de Lógica de Negócio (Pentest)
 - Como Testar: Tentar aplicar um cupom de desconto de 100% duas vezes ou alterar o preço de um item no carrinho via interceptação de requisição.

- Objetivo: Encontrar falhas que dependem do fluxo humano e que ferramentas automáticas ignoram.

Desafio final:

9. Como a falta de validação de input de dados na camada de aplicação pode expor um sistema a injeções de código malicioso, como SQL Injection e Cross-Site Scripting (XSS)?

A falta de validação de input permite que os dados enviados pelo usuário sejam interpretados e executados como código pelo sistema, quebrando a barreira que deveria existir entre o que é um "dado" e o que é uma "instrução".

1. SQL Injection (SQLi)

- Como ocorre: A aplicação concatena o texto do usuário diretamente na formação de uma consulta (query) ao banco de dados.
- A Injeção: O atacante insere caracteres de controle SQL (como ' , -- ou cláusulas como `OR 1=1`).
- O Impacto: O banco de dados interpreta a entrada como parte do comando executável. O atacante consegue manipular a lógica da query para contornar telas de login, roubar, alterar ou apagar dados confidenciais.

2. Cross-Site Scripting (XSS)

- Como ocorre: A aplicação recebe o texto do usuário e o renderiza na interface web para outros usuários sem neutralizar caracteres especiais.
- A Injeção: O atacante insere trechos de código executável pelo navegador (geralmente JavaScript, como `<script>... </script>`) em campos públicos, como comentários de um fórum.
- O Impacto: Quando vítimas acessam a página, seus navegadores executam o script malicioso. Isso permite o roubo de cookies de sessão (sequestro de conta) ou a execução de ações não autorizadas em nome do usuário logado.

Soluções Imediatas

- Para SQLi: Utilizar Prepared Statements (Consultas Parametrizadas), que forcem o banco de dados a tratar o input estritamente como dado isolado, não como parte do comando.
- Para XSS: Aplicar Output Encoding (Escapamento de Dados) antes de exibir qualquer informação na tela, convertendo caracteres de código (ex: `<`) em entidades seguras (ex: `<`;) para que o navegador exiba apenas o texto visual.